
A4: DEPENDENCY PARSING

Natural Language Processing
Vrije Universiteit Amsterdam

Group 63

Dheeraj Varghese²⁷⁵⁹⁵¹⁵, Manuel G. Scanferla²⁷⁶⁴³⁴⁸ & Ángel A. Gil²⁷⁸⁸⁹⁸⁸

April 26, 2023

Contributions

Each team member worked closely together on the assignment. We all tried to help each other with every question and worked collaboratively to create a polished final report as well as a clean code. We would like the marks to be equally distributed among us, as we all share responsibility for the strengths and weaknesses of our work.

1 Machine Learning & Neural Networks

(a) Stochastic Gradient Descent

In 3-4 sentences, please explain *stochastic* gradient descent and how it could be useful for an NLP task.

Stochastic gradient descent (SGD) is a variant of the gradient descent optimization algorithm. Unlike traditional gradient descent, which computes the gradient using the entire training dataset, SGD updates the model parameters using a random subset (or a single sample) of the training data, commonly referred to as *batch* or *minibatch*, at each iteration [1]. This randomness introduces noise, but it allows for faster convergence and makes SGD particularly useful for large-scale NLP tasks where the training data can be massive, as it enables the model to process and learn from smaller, manageable batches of data at a time.

(b) Adam Optimizer

i. Briefly explain in 2–4 sentences (you don’t need to prove mathematically, just give an intuition) how using m stops the updates from varying as much and why this low variance may be helpful to learning, overall.

Think about $\mathbf{m}$ in the Adam optimization algorithm as a savvy guide on a hike. Instead of reacting to every little change in the slope (which is what raw gradients would do), $\mathbf{m}$ takes a look at the last few steps, smooths out the path, and decides on a balanced, average step size. This way, we avoid erratic leaps or cautious tiptoes based on short-term, noisy gradients. Ultimately, this steady approach gets us to the bottom of the hill, or the minimum of our loss function, in a more reliable and often quicker way [2].

ii. Since Adam divides the update by $\sqrt{v}$, which of the model parameters will get larger updates? Why might this help with learning?

The $\mathbf{v}$ term in the Adam optimization algorithm is an estimate of the variance of the gradients. By dividing the update by $\sqrt{v}$, Adam effectively normalizes the update step by an estimate of its expected magnitude. In practice, this means that parameters associated with smaller, more consistent gradients (i.e., small $\mathbf{v}$ values) will get larger updates (because their updates are divided by a smaller number). Conversely, parameters with high-variance gradients (large $\mathbf{v}$ values) will get smaller updates [3].

This approach is beneficial because it allows for adaptive learning rates. If a certain parameter’s gradient doesn’t change much, it implies we’re likely in a flatter region for that parameter, and we can afford to take larger steps. But if a parameter’s gradient is very noisy, we are likely in a volatile region, and it’s safer to take smaller steps. This adaptivity can speed up convergence and improve the robustness of the learning process.

(c) Dropout

i. Why should dropout be applied during training?

Dropout is applied during training to prevent over-fitting and improve the generalization ability of the model. By randomly setting units in the hidden layer to zero, dropout intro-

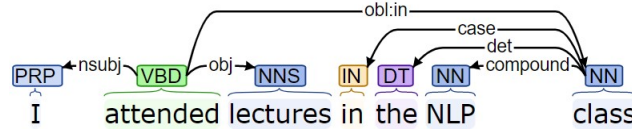
duces noise and forces the model to rely on different subsets of features during each training iteration. This helps to reduce the co-adaptation of neurons and encourages the model to learn more robust and generalized representations of the data [4].

ii. Why should dropout NOT be applied during evaluation?

Dropout should not be applied during evaluation (or testing) because it introduces randomness and modifies the expected behavior of the model. During evaluation, we want the model to make consistent and deterministic predictions based on the learned parameters. Therefore, dropout is typically turned off during evaluation to obtain reliable predictions.

2 Neural Transition-Based Dependency Parsing

(a) Go through the sequence of transitions needed for parsing the sentence “I attended lectures in the NLP class”.



Here is our sequence (note that we substituted *attended* by *att* and *lectures* by *lec* due to space limitations):

Stack	Buffer	New Dependency	Transition
[r]	[I, att, lec, in, ...]		Initial Conf.
[r, I]	[att, lec, in, ...]		SHIFT
[r, I, att]	[lec, in, the, ...]		SHIFT
[r, att]	[lec, in, the, ...]	att → I	LEFT-ARC
[r, att, lec]	[in, the, NLP, class]		SHIFT
[r, att]	[in, the, NLP, class]	att → lec	RIGHT-ARC
[r, att, in]	[the, NLP, class]		SHIFT
[r, att, in, the]	[NLP, class]		SHIFT
[r, att, in, the, NLP]	[class]		SHIFT
[r, att, in, the, NLP, class]	[]		SHIFT
[r, att, in, the, class]	[]	NLP → class	LEFT-ARC
[r, att, in, class]	[]	the → class	LEFT-ARC
[r, att, class]	[]	in → class	LEFT-ARC
[r, att]	[]	att → class	RIGHT-ARC
[r]	[]	r → att	RIGHT-ARC

(b) A sentence containing n words will be parsed in how many steps (in terms of n)? Briefly explain in 1–2 sentences why.

A sentence containing n words will be parsed in $2n$ steps (or $2n + 1$ if we take into account the initial state). This is because each word requires a SHIFT operation to move it from the buffer to the stack, and then either a LEFT-ARC or RIGHT-ARC operation to create a dependency and remove it from the stack, each of which counts as a separate step.

(c) Inspect the data you will be using manually. What format is it in? How is it separated? Explain how the data is labeled already for your use.

The dataset contains one text file and six Conference on Computational Natural Language Learning (CoNLL) format files. The text file contains pretrained word embeddings and the data is separated by whitespace or tabs. For our use each line contains the word followed by its embedding vector values. The word and the embedding vector values are separated by whitespace or tabs as well. On the other hand, in the CoNLL files each word in a sentence is represented by a separate line, and the lines are organized into columns separated by tab characters. The data in these files is labeled with linguistic annotations such as word index, word form (e.g., influential, members), POS tag (e.g., ADJ, NOUN, VERB), coarse-grained POS tag (e.g., JJ, NNS, VBD), dependency head index, and dependency relation (e.g., amod, nsubj, det). The labels indicate the linguistic properties and syntactic structure of the words in the sentences. Some columns may contain underscores (.) indicating missing or unavailable information for a particular word.

(d) Implement the init and parse step functions in the PartialParse class.

Implemented and commented in *parser_transitions.py*.

(e) Implement the algorithm for minibatch parse function in parser transitions.py.

Implemented and commented in *parser_transitions.py*.

(f) Train a neural network to predict, given the state of the stack, buffer, and dependencies, which transition should be applied next. The model will be trained to minimize cross-entropy loss and the evaluation metric used will be UAS (Unlabeled Attachment Score).

Implemented and commented in *parser_model.py* and *run.py*.

(g) Report the best UAS your model achieves on the dev set and the UAS it achieves on the test set in your write-up. Why is UAS a useful metric for evaluating dependency parsing? Cite specific examples where the UAS metric does and does not provide meaningful insight into the performance of your parser.

As shown in the snippet below (1) the best UAS score for the dev set is 88.66 with a loss of 0.067, and 89.11 for the test set.

This evaluation metric is useful because if a dependency parser consistently achieves a high UAS, it indicates that it is accurately identifying the syntactic relationships between words in a sentence, which is essential for understanding sentence structure and semantics.

The UAS metric provides meaningful insights for sentences containing homonyms or words with multiple possible dependencies, since if the score is low it suggests that the parser struggles to disambiguate between different syntactic relationships. The same could happen for sentences with complex coordination structures, where the low UAS score helps us prioritize the parser's difficulties.

```

Average Train Loss: 0.06697000300076295
Evaluating on dev set
1445850it [00:00, 28352054.94it/s]
- dev UAS: 88.66
New best dev UAS! Saving model.

=====
TESTING
=====
Restoring the best model weights found on the dev set
Final evaluation on test set
2919736it [00:00, 55132540.31it/s]
- test UAS: 89.11
Done!

```

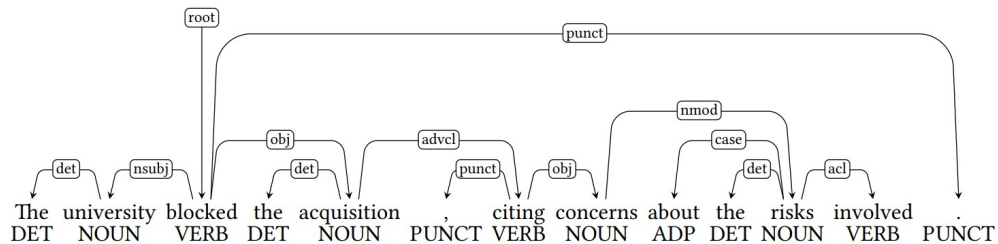
Figure 1: UAS scores

This evaluation metric sometimes could also "trick" us. For instance, in languages with strict word order like English, a parser might achieve a high UAS simply by exploiting the word order, without necessarily having a deep understanding of syntax. Since we are using this language in our parser this could be the case. Moreover, if the goal is to assess semantic accuracy or capture more nuanced linguistic phenomena this score doesn't provide useful insights. It also does not consider the parser's ability to capture contextual dependencies or resolve pronouns references across multiple sentences. This means that a parser may achieve a high UAS score while struggling to maintain coherent discourse-level understanding

3 Error Analysis

(a) For each sentence, state the type of error, the incorrect dependency, and the correct dependency. While each sentence should have a unique error type, there may be multiple possible correct dependencies for some of the sentences.

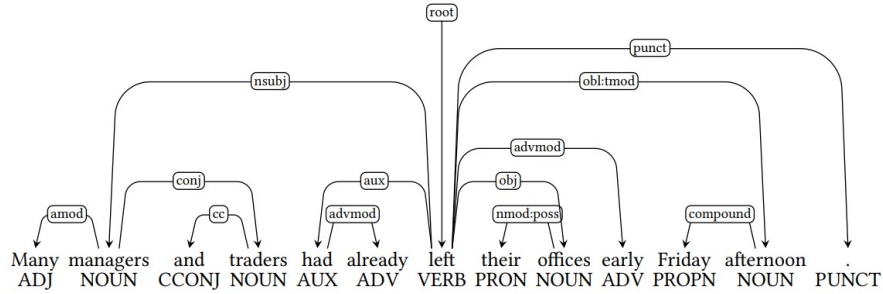
i.



- **Error type:** Verb Phrase Attachment Error

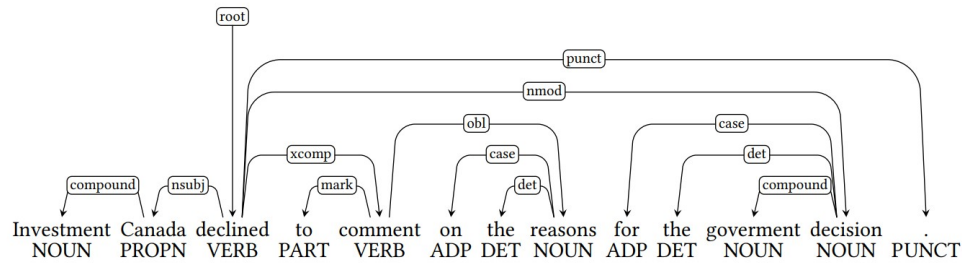
- **Incorrect dependency:** acquisition → citing
- **Correct dependency:** blocked → citing

ii.



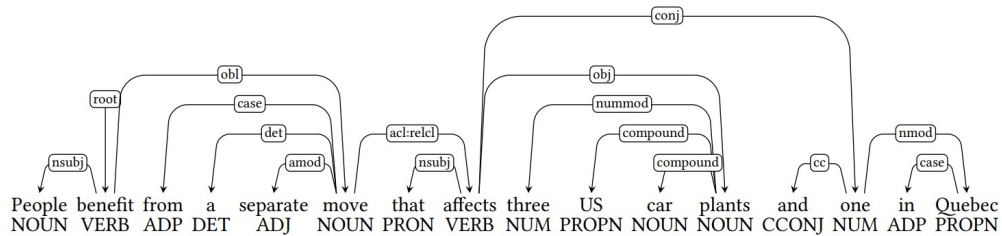
- **Error type:** Modifier Attachment Error
- **Incorrect dependency:** left → early
- **Correct dependency:** (Friday) afternoon → early

iii.



- **Error type:** Prepositional Phrase Attachment Error
- **Incorrect dependency:** declined → decision
- **Correct dependency:** reasons → decision

iv.



- **Error type:** Coordination Attachment Error
- **Incorrect dependency:** affects $\rightarrow$ one
- **Correct dependency:** plants $\rightarrow$ one

(b) Using an online dependency parser (e.g. CoreNLP), show (i) the initial “garden path” parse structure computed before the parse is corrected, and (ii) the final, correct parser of the sentence. Having now worked with a transition-based dependency parser, how do you think it would handle such sentences?

Garden-path sentences, like “The horse raced past the barn fell,” can pose significant problems for a dependency parser due to their initial syntactic ambiguity. They lead the reader (or, in our case, the parser) down a ‘garden path’ to an unexpected grammatical structure, which often necessitates a reevaluation of the parsed sentence structure.

In this sentence, the issue stems from the phrase “raced past the barn,” which seems to be a complete and grammatical thought with ‘horse’ as the subject and ‘raced’ as the verb. This is known as a “reduced relative clause,” where ‘that was’ is omitted from the sentence, and the correct interpretation should be “The horse (that was) raced past the barn fell.”

In this regard, there is nothing wrong with the parsing of (i), the initial “garden path” parse structure computed by CoreNLP (Figure 2). The problem arises in (ii), the final structure computed by CoreNLP (Figure 3). Our insight is that, when initially parsed, the parser might incorrectly attach ‘raced’ as the main verb of the sentence, with ‘horse’ as its subject, since this is a common pattern. This will result in a problem when the parser reaches ‘fell’ and tries to incorporate it into the structure because ‘horse’ already has a verb, ‘raced’. However, if the parser correctly identifies ‘raced’ as part of the reduced relative clause, it can correctly parse the sentence. ‘Raced past the barn’ modifies ‘horse,’ and ‘fell’ is the main verb with ‘horse’ as the subject. We could not find a parser who performed it right, so we corrected the parsing manually (Figure 4).

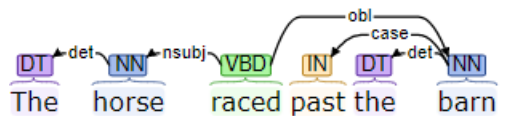


Figure 2: Initial “garden path” by CoreNLP

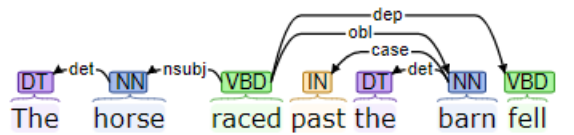


Figure 3: Final “garden path” by CoreNLP

To parse these garden-path sentences correctly, the parser should have the ability to manage such syntactic uncertainty. Ideally, this would be achieved by training it on a dataset that contains a substantial number of reduced relative clauses. However, such sentences are not commonly found, which may lead to the parser encountering difficulties if

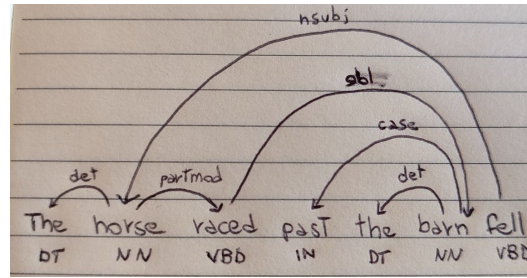


Figure 4: Manually parsed "garden path"

it has been predominantly trained on more standard sentence structures. Additionally, the ambiguity's resolution can be computationally demanding, especially if the parser is set up to make greedy decisions or operate without the option to reconsider past decisions.

(c) Explain the benefit of using part-of-speech tags as features in the parser. You may cite examples from your parser and garden-path sentences as support.

Using part-of-speech tags as features in the parsing process has several advantages, some of which regard for instance disambiguation, syntactic relations and a better handling of "garden-path" sentences.

Sometimes a single word can function in different roles depending on its context. For instance, in the sentence "I saw a man with a telescope," the word "saw" could be interpreted as a past-tense verb or as a noun. POS tagging helps disambiguate this kind of sentences by providing explicit information about a word's role in the sentence. Moreover, POS tags can help predict transitions and relations by exploiting the probable syntactic relations that should exist between words. For example, a verb usually follows a subject noun phrase. In the context of our algorithm, the neural network could use these POS tags as part of its input features and that would help the model make better decisions in complex situations, like resolving ambiguities, deciding the next transition to apply or dealing with long-distance dependencies. Furthermore, POS tags can be helpful for the parser to interpret "garden-path" sentences. As we noticed in the previous answer, CoreNLP had difficulties in correctly parsing "The horse raced past the barn fell". However, if it knew that "raced" is a past-participle verb (not a past-tense verb), we can suppose that it could have parsed the sentence correctly as "The horse (that was) raced past the barn fell".

4 Bonus: Cross-lingual dependency parsing

Choose a language you are familiar with other than English, and explore how the four types of parsing errors introduced in (3a) may be challenges in cross-lingual dependency parsing. To do this, follow these steps:

- Choose an example English sentence that demonstrates each error type.
- Translate these sentences into the language you are working with. Explain your translation process.
- Run a dependency parser for your language on the translated sentences.
- Assess how the challenges of English dependency parsing using a transition-based parser translate into the language you have chosen. Are the challenges the same? Are there new challenges that English may not face?

Language chosen: Spanish

We decided to stick to the example sentences from 3 a) for this section because they were already employed to illustrate each type of error. This way, we are certain that they are correct and, besides, they are straightforward, which makes it easier to focus on the error type at hand.

Original English Sentences with Corresponding *Parsing Error type* and explanation:

1. **Moscow sent troops into Afghanistan.** *Prepositional Phrase Attachment Error*: The phrase *into Afghanistan* is a prepositional phrase. In this example, *sent* is the correct head word, so attaching *into Afghanistan* to any other head word such as *troops* is a mistake.
2. **Leaving the store unattended, I went outside to watch the parade.** *Verb Phrase Attachment Error*: The phrase *leaving the store unattended* is a verb phrase. In this example, the correct head word is *went*, so attaching *leaving the store unattended* to a different head word would lead to the aforementioned error.
3. **I am extremely short.** *Modifier Attachment Error*: The adverb *extremely* is a modifier of the adjective *short* (i.e., *short* is the correct head for *extremely*). Attaching *short* to any other head word would not be right.
4. **Would you like brown rice or garlic naan?** *Coordination Attachment Error*: The phrases *brown rice* and *garlic naan* are both conjuncts and the word *or* is the coordinating conjunction. The second conjunct (*garlic naan*) should be attached to the first conjunct (*brown rice*) otherwise we would have the aforementioned error.

Spanish translation by ChatGPT

1. Moscú envió tropas a Afganistán.
2. Dejando la tienda sin atención, salí a ver el desfile.

3. Soy extremadamente bajo (or "baja" for a female speaker).
4. ¿Te gustaría arroz integral o naan de ajo?

Spanish translation by a native speaker

1. Moscú envió tropas a Afganistán.
2. Dejando la tienda **desatendida**, salí a ver el desfile.
3. Soy extremadamente bajo (or "baja" for a female speaker).
4. ¿**Quieres** arroz integral o naan de ajo?

Translation process and comparison

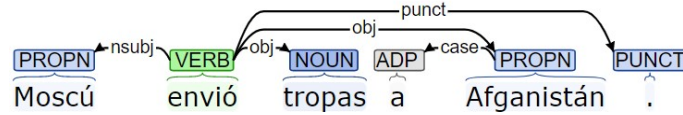
ChatGPT claim, to which I also adhere: "These translations were generated while keeping in mind the principles of Spanish syntax and vocabulary. An effort was made to retain the potential for the same types of attachment errors as in the original English sentences." In this case, the LLM does a neat job with the syntactic structure: the difference is nonexistent between the AI and the human translations. This might be because for these 4 simple examples, there are similar structures in Spanish which carry the same information, so the translation can be performed practically word by word.

The only change that a native Spanish speaker would consider regarding syntax is re-ordering the second sentence to "Salí a ver el desfile dejando la tienda desatendida", since it feels more natural. Nevertheless, there is nothing wrong with the original order and it might be "better" from an academic point of view. That is because allocating an object first in a sentence usually implies that this object carries the most important information (in the sentence). Since the fact that "the store was left unattended" seems to be the most important part of the sentence, it is convenient to mention it first (although it could sound slightly pretentious in spoken Spanish).

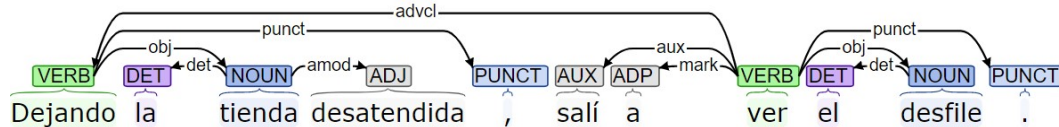
Regarding vocabulary, there are a couple of minor mistakes (anyone would understand the sentences, but there are more suitable ways to put it). Both "Sin atención" and "desatendida" mean "unattended" in Spanish, but with the first phrase, the sentence would literally be read "Leaving the store *with no attention*, ..." in English. Even "Sin atender" works slightly better than "sin atención", but the latter is not wrong per se. A similar phenomenon occurs in the second correction, "Te gustaría" against "Quieres". Although the first one is the literal translation, the second one is more suitable for a similar reason. Compared to English, there are (several) additional verb tenses in Spanish. "Would", which represents the subjunctive mood (i.e. how you express hypothetical situations and outcomes) is represented by a whole different conjugation. In this case, "Te gustaría" is in subjunctive mode. This is a correct alignment of "would" with the Spanish subjunctive by ChatGPT, but it gets more subtle than that. We do not use "would you like" like this in Spanish, we use "do you want", because "like" for us is only used enjoying an activity or simply *liking* something, not *wanting* something. Although "would you like" is a polite way to say "do you want" in English, there is no such differentiation in Spanish. We could say "Querías?", which translates to "Would you want", but the subjunctive here doesn't feel natural.

Dependency parse of the translated sentences with CoreNLP

i.



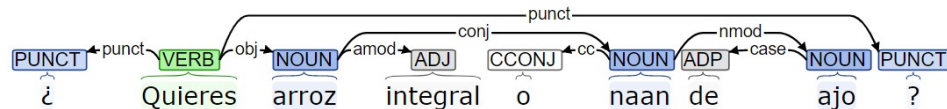
ii.



iii.



iv.



How the challenges of English dependency parsing using a transition-based parser translate into Spanish

Spanish parsing can also present similar challenges to those of English. Lets discuss the examples one by one:

1. **Prepositional Phrase Attachment Error:** In the sentence "Moscú envió tropas a Afganistán", the parser could have mistakenly attached "a Afganistán" to "tropas" instead of "envió". Nevertheless, the parser did not make any mistake with this sentence.
2. **Verb Phrase Attachment Error:** In "Dejando la tienda desatendida, salí a ver el desfile", the parser incorrectly attached "Dejando la tienda desatendida" to "ver" instead of "salí", just as it could happen in English.
3. **Modifier Attachment Error:** In "Soy extremadamente bajo", the parser could incorrectly attach "extremadamente" to "Soy" instead of "bajo", although it did not happen this time.
4. **Coordination Attachment Error:** In "¿Quieres arroz integral o naan de ajo?", a parser might incorrectly attach "naan de ajo" to "quieres" instead of "arroz integral". It did not happen this time, but on a previous run (i did not take a picture) the sentence was parsed wrongly (I swear).

Additional challenges in Spanish may arise due to varying word order compared to English and the use of pronominal words that are attached to the verb (e.g., "lo se", which means

”I know it/that”). The use of reflexive and indirect object pronouns, which usually precede the verb, can add complexity for parsers. Furthermore, Spanish’s extensive verb conjugation system to express tense, aspect, mood, and subject can make identifying the correct heads of certain relations a more intricate task. For instance, in a language where the subject is usually explicitly stated as a separate word, parsers can sometimes rely on word order to identify the subject of a verb. But in Spanish, the subject is often implicit within the verb conjugation itself. This can complicate the parser’s job, as it needs to infer the subject from the verb form, and it can lead to errors if the parser does not handle this feature of Spanish effectively.

References

- [1] Y. Bengio, “Practical recommendations for gradient-based training of deep architectures,” *Lecture Notes in Computer Science*, p. 437–478, 2012.
- [2] “Cs231n convolutional neural networks for visual recognition.” [Online]. Available: <https://cs231n.github.io/neural-networks-3/#sgd>
- [3] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *arXiv preprint arXiv:1412.6980*, 2017.
- [4] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” *J. Mach. Learn. Res.*, vol. 15, no. 1, p. 1929–1958, 2014.